Threads

Universidade Estadual de Mato Grosso do Sul
Bacharelado em Ciência da Computação
Sistemas Operacionais
Prof. Fabrício Sérgio de Paula

Tópicos

- Conceitos
- Modelos multithreads
- Bibliotecas
- Opções de criação de threads
- Threads do Windows XP
- Threads do Linux

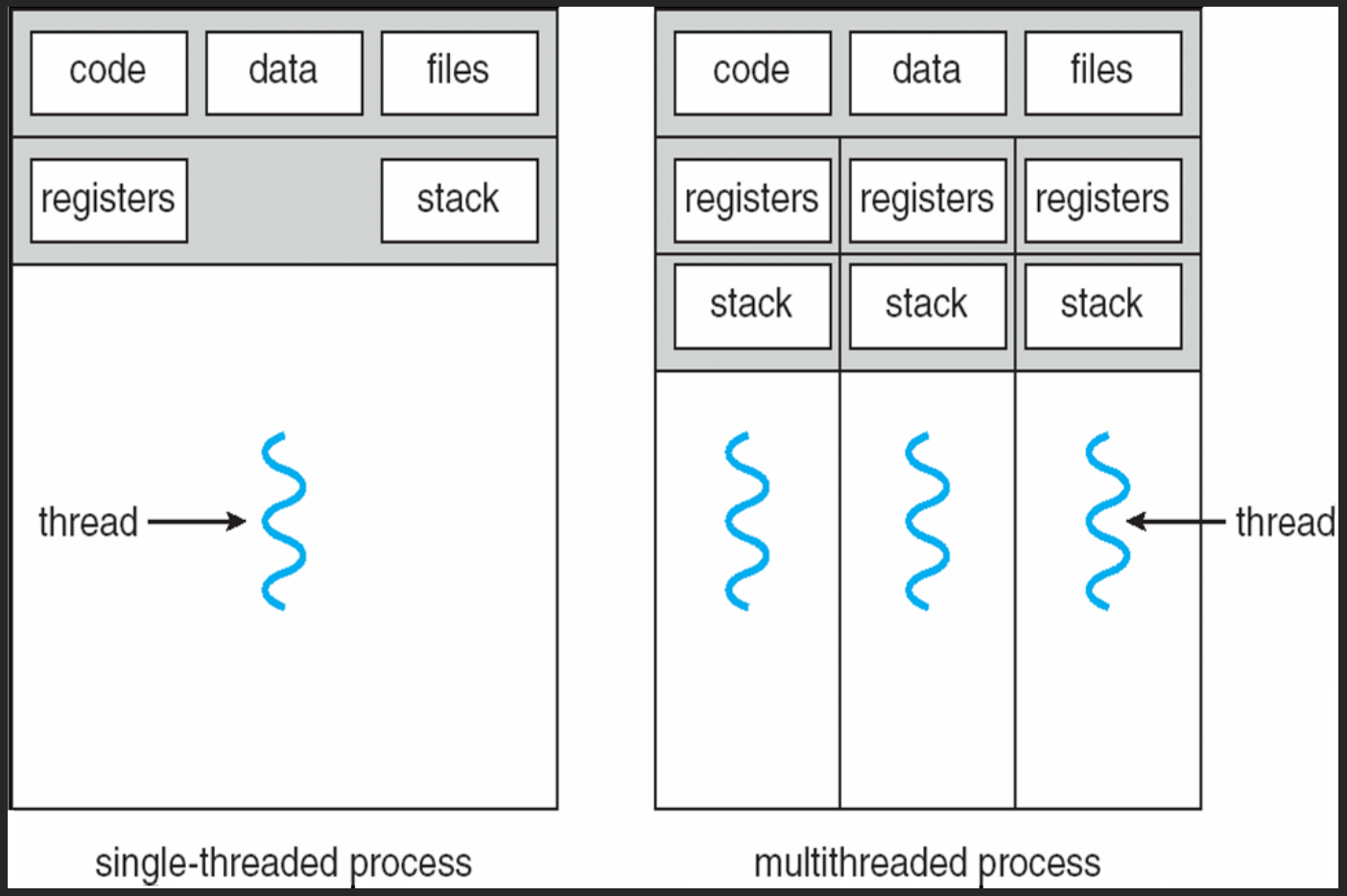
Conceitos

- Thread: unidade básica de utilização da CPU
  - Possui
    - Identificação (ID)
    - Contador de programa (PC)
    - Conjunto de registradores
    - Pilha
  - Compartilha com threads de mesmo processo
    - Seção de código e dados
    - Outros recursos: arquivos abertos, sinais

Conceitos

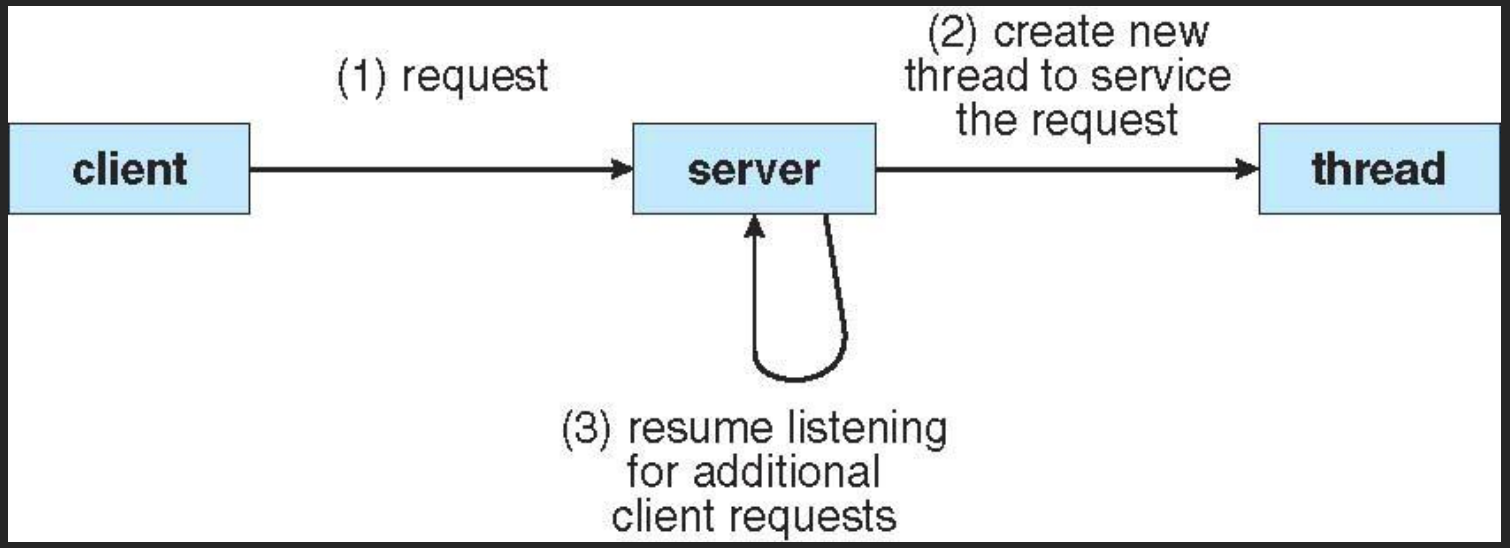
- Processo tradicional (heavyweight): uma única thread de controle (singlethread)
- Processo com múltiplas threads de controle (multithread): pode executar mais de uma tarefa simultaneamente
- Aplicações atuais: multithread
  - Ex.:
    - Browser: uma thread para mostar texto/imagens outra para obtenção dos dados na rede
    - Processador de texto: uma thread para mostrar imagens, outra para tratar teclas digitadas, outra para verificação ortográfica

Conceitos


code
data
files
registers
stack
thread
single-threaded process
code
data
files
registers
registers
registers
stack
stack
stack
thread
multithreaded process

Conceitos

- Uso de threads em sistemas cliente-servidor:



Conceitos

- Kernels de SOs atuais são multithread
  - Várias threads operam em modo kernel: cada realizando tarefa específica
  - Ex.:
    - Solaris: usa conjunto de threads especificamente para tratamento de interrupções
    - Linux: usa uma thread para gerenciar memória disponível no sistema

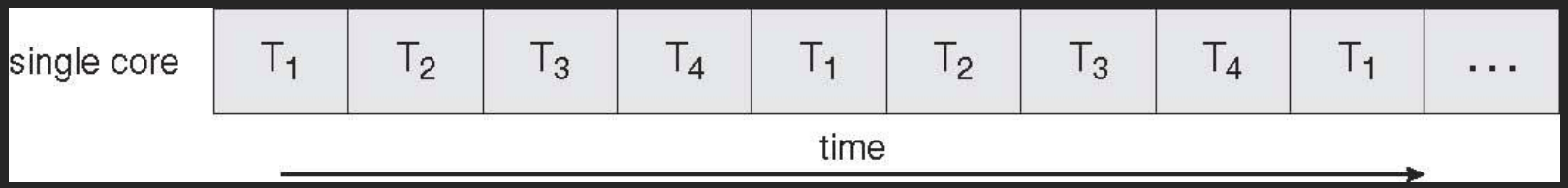
Conceitos

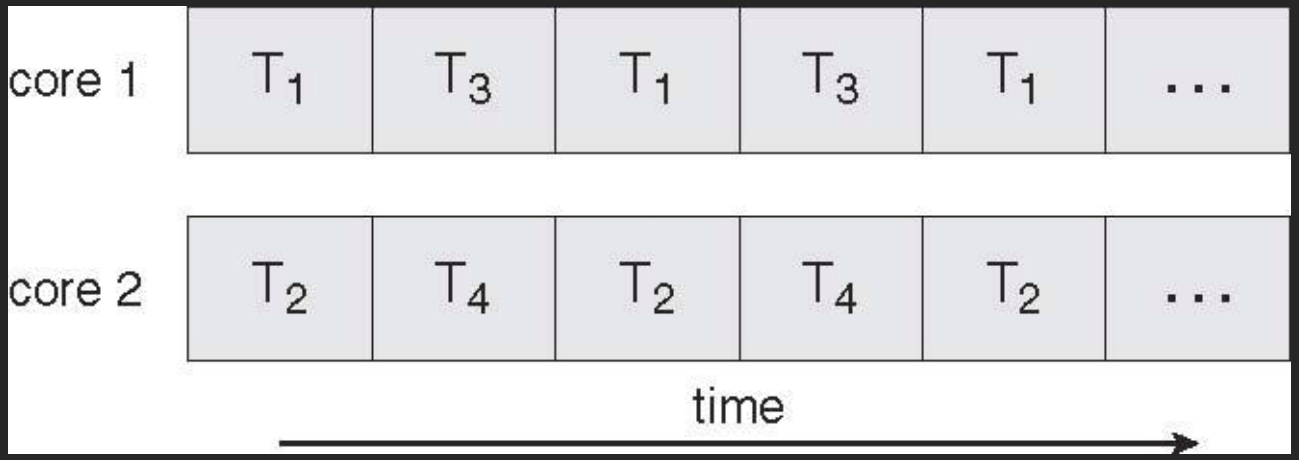
- Benefícios
  - Capacidade de resposta: menor tempo de resposta
    - Thread interativa continua execução, mesmo que parte da aplicação esteja bloqueada aguardando outra operação
  - Compartilhamento de recursos:
    - Por default: código, memória e recursos do processos
  - Economia:
    - Compartilhamento: menor uso de memória
    - Criação e troca de contexto mais econômica/rápida
      - Solaris 2: criação 30x mais rápida e troca de contexto 5x
  - Escalabilidade: processo single-thread executa em um único processador, independente da quantidade disponível
    - Multithread: incrementa o paralelismo em máquina multi-CPU

Conceitos

- Hardwares atuais: multiplos cores em um único chip
  - Programação multithread: uso eficiente de múltiplos cores e melhor concorrência
  - Ex.: aplicação com 4 threads
    - Um core: execução das threads intercadada
    - Vários cores: theads são executadas em paralelo

Conceitos


single core
T_1 T_2 T_3 T_4 T_1 T_2 T_3 T_4 T_1 ...
time


core 1
T_1 T_3 T_1 T_3 T_1 ...
core 2
T_2 T_4 T_2 T_4 T_2 ...
time

Conceitos

- Tendência de sistemas multicore pressiona desenvolvedores de SOs e aplicações: uso eficiente do hardware
  - SOs: algoritmos de escalonamento para usar múltiplos cores em paralelo
  - Aplicações: desafio é modificar programas existentes para novos multithread

Conceitos

- Dificuldades com programação multithread:
  - Dividir atividades: identificar quais partes da aplicação podem ser separadas e executadas com tarefas em paralelo
  - Balanceamento: garantir que tarefas tenham trabalho de "igual valor"
  - Divisão de dados: separação dos dados acessados em cada tarefa separada
  - Dependência de dados: tarefas podem necessitar acessar dados das outras
    - Necessidade de sincronização
  - Teste e depuração: há multiplas formas de execução em paralelo, dificultando o processo de teste e depuração

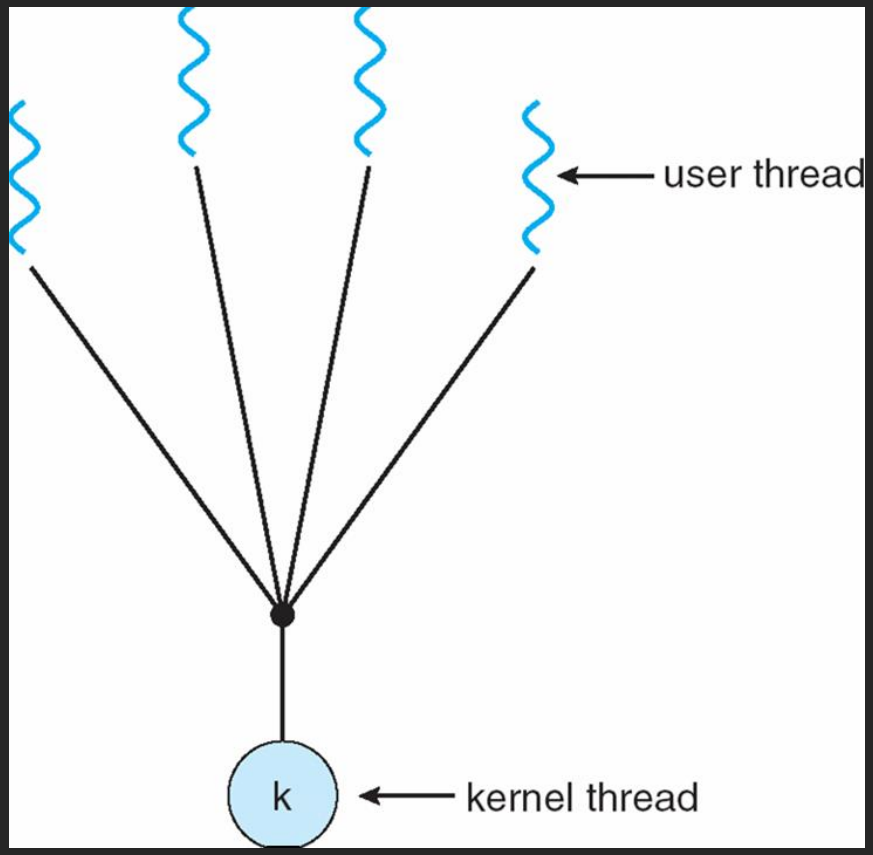
Modelos multithreads

- Suporte a threads pode ocorrer no nível do usuário (theads de usuário) ou no nível do kernel (threads de kernel)
  - Threads de usuário: suportadas “acima do kernel” e gerenciadas sem suporte do kernel
    - Biblioteca dá o suporte e gerencia
  - Threads de kernel: suportadas e gerenciadas pelo kernel
    - Todos os SOs contemporâneos suportam
- Modelos multithreads: relacionam threads de usuário às de kernel
  - Tipos: Muitos-para-um, um-para-um e muitos-para-muitos

Modelos multithreads

- Muitos-para-um:
  - Muitas threads no nível do usuário associadas a uma única thread de kernel
    - Gerenciamento é realizado no espaço de usuário
    - Se uma thread faz chamada ao sistema, o processo todo fica bloqueado
    - Só uma thread tem acesso ao kernel por vez: não funciona em paralelo
      - Nem em sistemas multiprocessados
  - Usado em sistemas que não suportam threads de kernel
  - Ex.: Solaris Green Threads, GNU Portable Threads

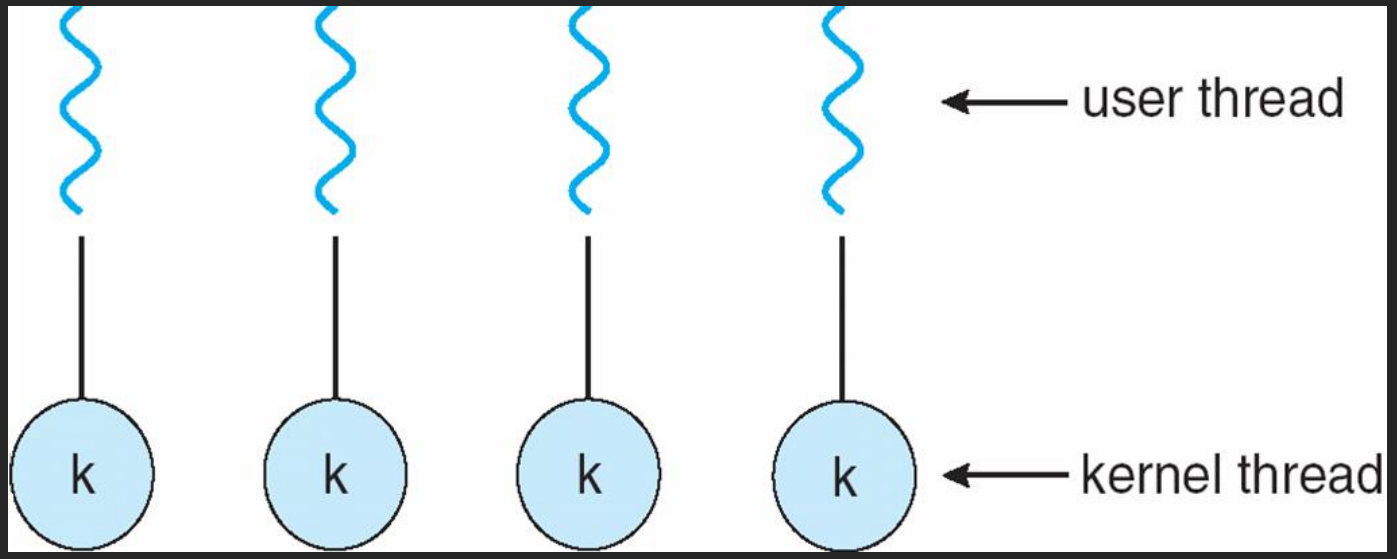
Modelos multithreads



Modelos multithreads

- Um-para-um:
  - Cada thread do usuário associada a uma thread de kernel
    - Possibilita mais concorrência que o muitos-para-um
    - Se uma thread é bloqueada outra pode executar
    - Permite usar os recursos paralelos
    - Mais overhead na criação
      - Cada thread de usuário requer a criação de uma correspondente no kernel
        - Quantidade de threads que podem ser criadas é restrita
  - Ex.: Windows NT/XP/2000, Linux, Solaris 9 em diante

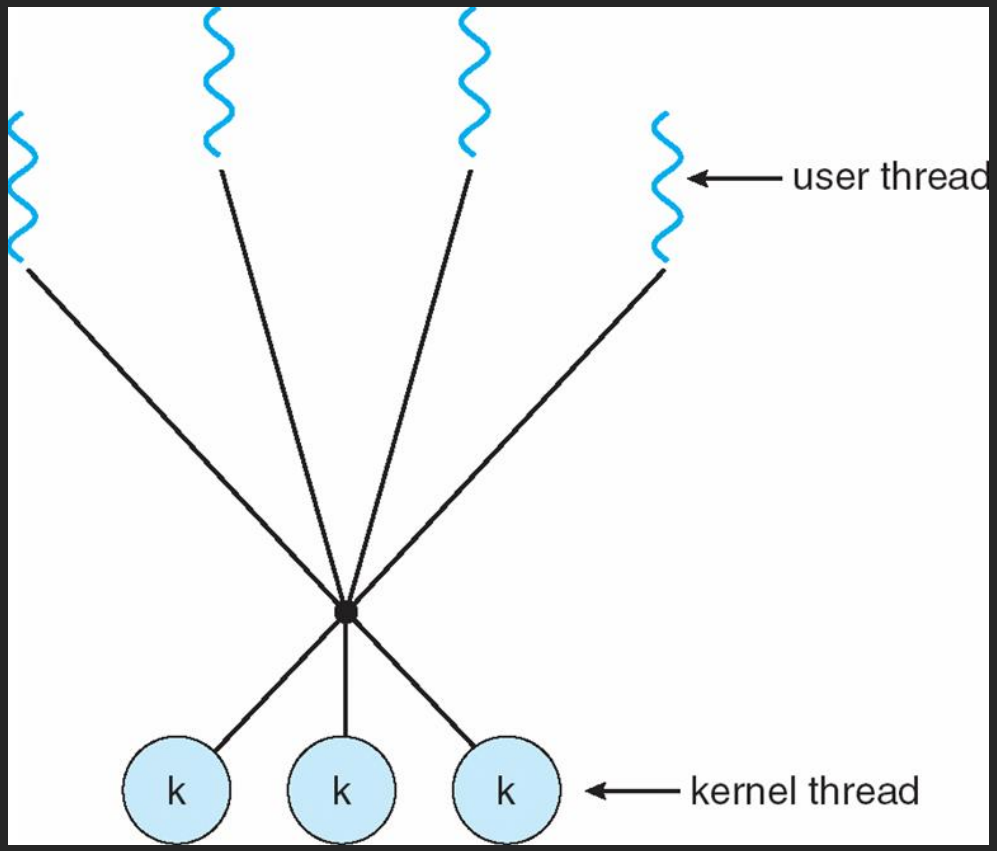
Modelos multithreads



Modelos multithreads

- Muitos-para-muitos:
  - Permite associar várias threads de usuário a um número menor ou igual de threads de kernel
    - Número de threads de kernel
      - Pode ser específico para aplicação ou para uma máquina
    - Programador pode criar tantas threads de usuário quanto desejar
      - Concorrência real: determinada pelo número de threads que o kernel suporta
        - Se o kernel só suportar apenas uma não há paralelismo
  - Ex.: Solaris antes da versão 9, Windows NT/2000 com pacote *ThreadFiber*

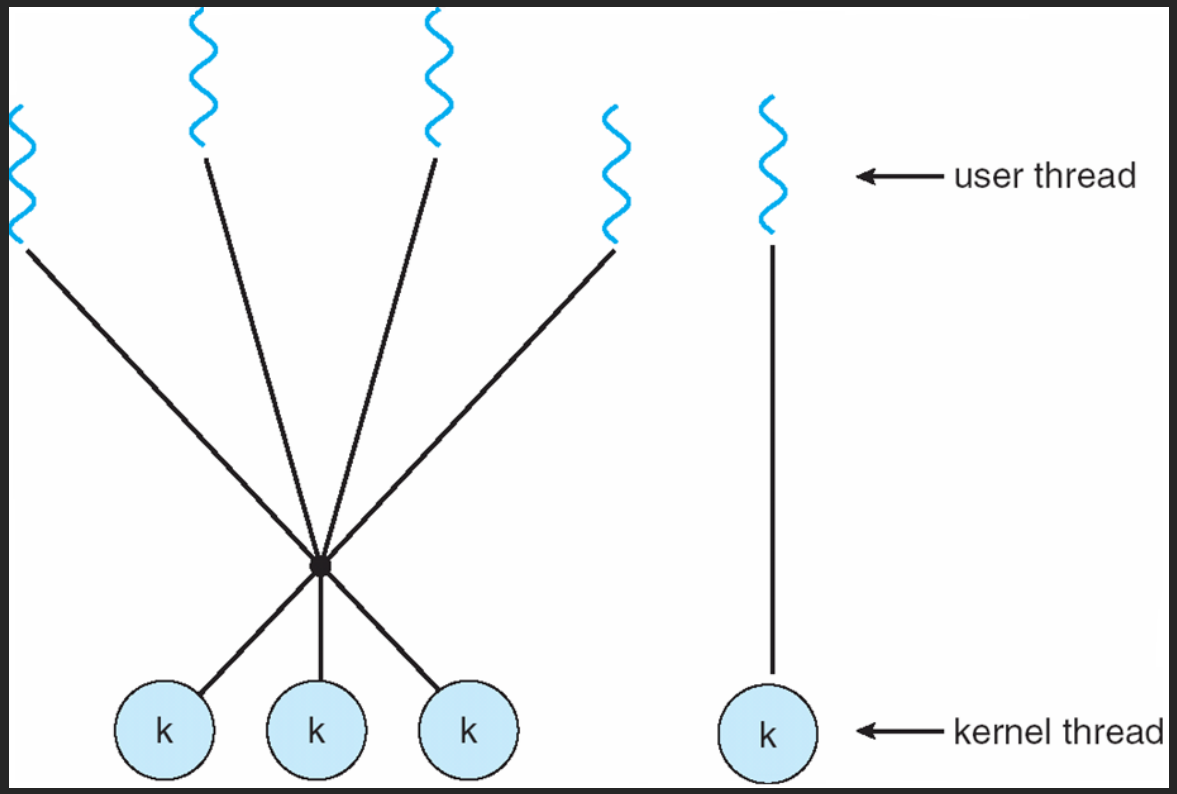
Modelos multithreads



Modelos multithreads

- Modelo de dois níveis:
  - Variação do muitos-para-muitos
    - Permite muitos-para-muitos
    - Permite, ainda, um-para-um
  - Ex.: IRIX, HP-UX, Tru64 UNIX, Solaris 8 e anterior

Modelos multithreads



Bibliotecas

- Biblioteca de threads: provê API para criar e gerenciar threads
  - Duas formas de implementar:
    - Biblioteca totalmente no espaço de usuário
      - API: funções locais
    - Biblioteca no nível do kernel suportada pelo SO
      - API: execução de chamadas ao sistema

Bibliotecas

- Principais bibliotecas:
  - POSIX Pthreads: padrão POSIX (IEEE 103.1c) para criação e sincronização
    - Pode ser oferecida como biblioteca no nível de usuário ou kernel
    - Comum nos sistemas UNIX: Solaris, Linux, Mac OS X
  - Win32: nível de kernel para sistemas Windows
  - Java: implementação usa biblioteca do sistema hospedeiro
    - No Windows: API é implementada usando API Win32
    - No Linux/UNIX: API usa Pthreads

Bibliotecas

- Exemplo Pthreads:
  - Programa a seguir recebe argumento da linha de comando que é um número N
  - Cria uma thread para calcular sum = $\sum_{i=0}^{N} i$
    - sum é variável compartilhada
  - Thread principal imprime sum calculada pela thread criada

Bibliotecas

```
#include < pthread.h >
#include < stdio.h >
int sum; /* this data is shared by the thread(s) */
void *runner(void *param); /* the thread */
int main(int argc, char *argv[])
{
   pthread t tid; /* the thread identifier */
   pthread attr t attr; /* set of thread attributes */
   if (argc != 2) {
      fprintf(stderr,"usage: a.out <integer value> \ n");
      return -1;
   }
   if (atoi(argv[1]) < 0) {
      fprintf(stderr,"%d must be >= 0 \ n",atoi(argv[1]));
      return -1;
   }
   /* get the default attributes */
   pthread attr init(&attr);
   /* create the thread */
   pthread create(&tid,&attr,runner,argv[1]);
   /* wait for the thread to exit */
   pthread join(tid,NULL);
   printf("sum = %d \ n",sum);
}

 /* The thread will begin control in this function */
void *runner(void *param)
{
   i nt i, upper = atoi(param);
   sum = 0;
   for (i = 1; i <= upper; i++)
      sum += i;
   pthread exit(0);
}
```

Opções de criação de threads

- Semântica de fork() e exec()
  - fork(): cria novo processo
    - Alguns sistemas UNIX têm duas versões de fork():
      - Duplica todas as threads
      - Duplica apenas a thread que fez fork()
  - exec(): substituem o processo inteiro, incluindo suas threads
    - fork() seguida de exec(): duplicação apenas da thread de chamada

Opções de criação de threads

- Cancelamento de thread:
  - Termina thread antes de sua execução ser completada
    - Ex: Várias threads cooperam em uma busca
      - Se uma encontra informação: demais podem ser canceladas
    - Thread-alvo: thread que irá ser cancelada
  - Dois métodos:
    - Cancelamento assíncrono: termina thread-alvo imediatamente
      - Pode gerar inconsistências de dados compartilhados
    - Cancelamento adiado: permite thread alvo verificar periodicamente se deve ser cancelada
      - Permite terminar corretamente, mantendo consistência

Opções de criação de threads

- Tratamento de sinais:
  - Sinais: usados para notificar um processo sobre evento
  - Sinal pode ser:
    - Síncrono
      - Se for liberado para o mesmo processo que provocou o sinal
        - Ex.: processo “executa” divisão por 0
    - Assíncrono
      - Se fore gerado por evento externo
        - Ex.: sinal enviado de um processo a outro

Opções de criação de threads

- (cont.):
  - Sinal para processos monothread
    - Liberado apenas para o processo específico (PID)
  - Sinal para processo multithread: várias opções
    - Liberado para a thread conveniente
      - Ex.: a que executou divisão por zero
    - Liberado para todas as threads do processo
      - Ex.: sinal para de término do processo
    - Liberado para determinadas threads
    - Designar uma thread específica para receber todos os sinais

Opções de criação de threads

- Cadeia de threads:
  - Criação/finalização de threads gera overhead
    - Ex.: servidor web que trata muitas conexões por segundo
      - Criar e terminar threads muito custoso
  - Solução: cadeia de threads em espera
    - Inicialização do processo: criada quantidade adequada de threads
    - Threads aguardam para entrar em funcionamento
    - Quando o servidor recebe solicitação
      - Desperta uma thread da cadeia (se houver disponível, senão espera) e repassa trabalho
    - Quando thread completa serviço, volta à cadeia de espera
  - Vantagens: mais rápido que criar thread, limita recursos
  - Número de threads: relacionado aos recursos disponíveis

Opções de criação de threads

- Dados específicos de threads:
  - Cada thread pode ter sua própria cópia dos dados
    - Útil em algumas situações
    - Ex. cadeia de threads:
      - Thread despertada deve operar dados independentes
  - Bibliotecas incluem suporte a essa capacidade (ex.: Win32 e PThreads)

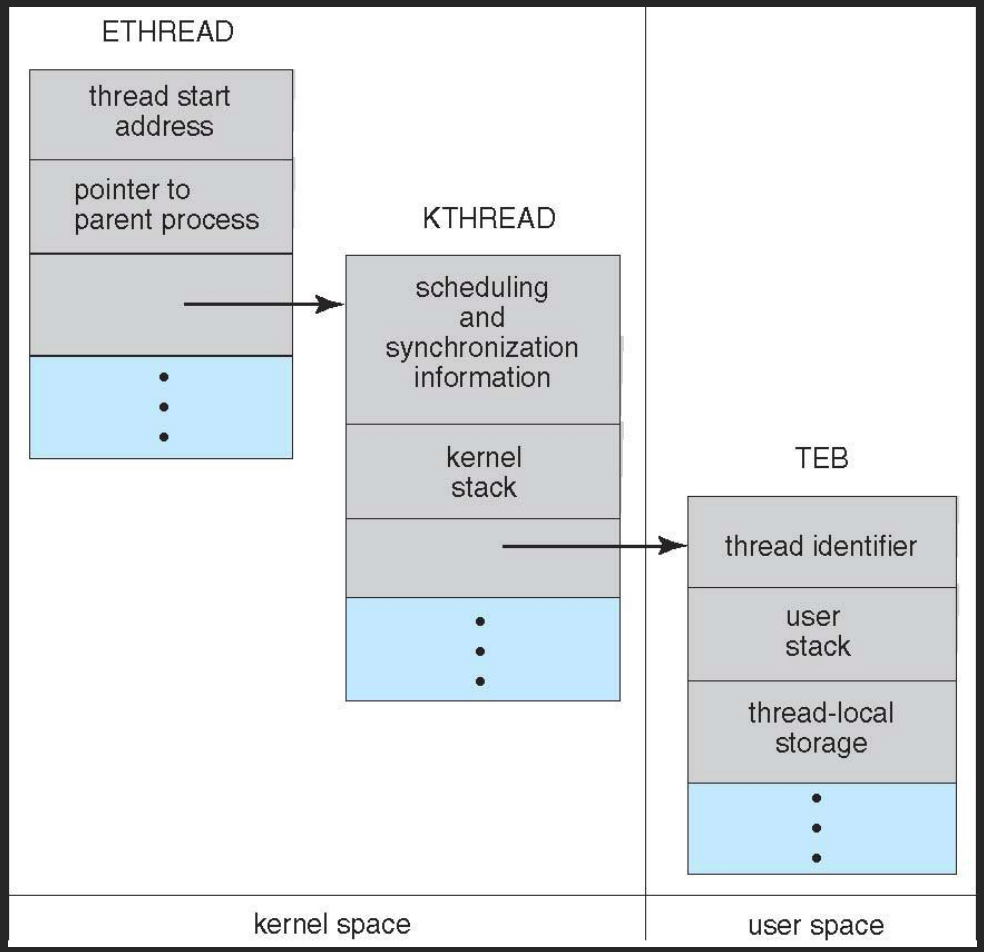
Threads do Windows XP

- Uma aplicação no Windows XP executa como um processo
  - Cada processo: uma ou mais threads
- Mapeamento: um-para-um
  - Mas pode ser muitos-para-muitos: bilbioteca fiber
- Componentes de uma thread:
  - Thread ID única
  - Conjunto de registradores
  - User stack (usada no modo usuário) e kernel stack (modo kernel)
  - Área de armazenamento privada: usada por várias DLLs

Threads do Windows XP

- Contexto da thread: composto pelo conjunto de registradores + user stack + kernel stack + armazenamento privado
- Principais estruturas de dados de uma thread:
  - ETHREAD (executive thread block): ponteiro para processo, endereço da rotina inicia, etc.
  - KTHREAD(kernel thread block): informações de escalonamento e sincronização, kernel stack, etc.
  - TEB (thread environment block): thread ID, user stack, dados específicos da thread, etc.
    - Somente TEB é armazenada no espaço de usuário

Threads do Windows XP



Threads do Linux

- Criação de processo no Linux: fork()
- Criação de thread: clone()
- Linux não distingue threads de processos: tudo é uma tarefa (task)
  - Task: fluxo de controle em um programa
  - Estrutura de dados: task_struct
- Chamada a clone(): flags indicam o que será compartilhado entre tarefas pai e filha
  - Compartilhamento: mesmo ponteiro (ex.: arquivos abertos) para estrutura de dados da task_struct do pai
  - Se nada é compartilhado, clone() equivale a fork()

Threads do Linux

flag	meaning
CLONE_FS	File-system information is shared.
CLONE_VM	The same memory space is shared.
CLONE_SIGHAND	Signal handlers are shared.
CLONE_FILES	The set of open files is shared.